

هدف الفصل

- 1- تعريف هندسة البرمجيات وأهميتها والأسس التي تقوم عليها.
- 2- الإجابة عن بعض الأسئلة حول هندسة البرمجيات.
- 3- التركيز على الجوانب الأخلاقية والمهنية وعلاقتها بهندسة البرمجيات.
- 4- تعرف نماذج الإجرائية البرمجية.
- 5- تعريف الأنشطة الأساسية في الإجرائية البرمجية.

1. مقدمة

من المعلوم أن اقتصاد جميع الدول المتقدمة أصبح اليوم مرتبطاً بالبرمجيات، وقد تزايد عدد الأنظمة التي تتحكم بها البرمجيات في عالمنا تزايداً ملحوظاً. كما أصبحت عائدات البرمجيات تشكل نسبة عالية من الدخل القومي في الدول المتقدمة. ومع ازدياد الجزء البرمجي في الأنظمة الحديثة، ظهرت عيوب وأخطاء تسببت فيها هــ. ذه الأـجـ. زاء. ويمكن الاطلاع على أمثلة عن بعض هذه الأخطاء فيما يلي:

1- خطأ أعوام القرن العشرين 1900.

في العام 1992 تلقت السيدة ماري من ولاية مينيسوتا دعوة للالتحاق بـروضة أطفال وهي تبـ. غـ مـ. ن العمر 104 سنة!

2- خطأ السنين الكبيسة.

غرم أحد المتاجر بمبلغ \$1000 بسبب احتفاظه باللحم يوماً إضافياً في 29 شباط 1988. ويعود الخطأ في ذلك إلى البرنامج الذي استخدم لطباعة تاريخ لنتهاء الصلاحية الذي لم يتنبه إلى أن العام 1988 هو سنة كبيسة.

3- تصميم سيء للواجهات.

في نيسان 1990 غادر قطار في لندن المحطة دون أن يكون السائق على متنه. فقد ضغط الـ. سائق زر الانطلاق لكن البرنامج كان مصمماً بحيث لا يتحرك القطار حتى تكون جميع أبوابه مغلقة. كان أحد الأبواب عالماً فنزل السائق ليحرره وما إن أغلق الباب حتى انطلق القطار دون سائقه.

4- الأمن.

ارتفع عدد الحوادث الأمنية التي أبلغ بها CERT¹ من 252 حادثاً في 1990 إلى 21756 حادثاً في 2000 وبلغ أكثر من 40000 حادثاً في 2001.

5- التسليم المتأخر وتجاوز الميزانية.

في العام 1995 تسببت الأخطاء الموجودة في نظام شحن الحقايب المعد لمطار دنفر الدولي الجديد - د بياتلاف الحقايب الصغيرة. أدى هذا الخطأ إلى تأخير افتتاح المطار لمدة 16 شهراً وإلحاق أضرار مالية بقيمة 3.2 مليار دولار وفي النهاية كانت معظم مراحل نظام شحن الحقايب يدوية.

6- التسليم ضمن المدة.

بعد 18 شهراً من التطوير تم تسليم نظام لشركة تأمين صحي في ويسكنسون بلغت كلفته 200 مليون دولار. غير أن النظام لم يكن يعمل بشكل صحيح! احتاج تعديله إلى دفع كاف وصلت إلى 60 مليون دولار واستغرق الأمر ثلاث سنوات!

7- التعقيد غير الضروري.

كلف طائفة شحن البضائع C-17 أكثر من 500 مليون دولار فوق ميزانيتها الأصلية بسبب أخطاء في النظام البرمجي للطيران. في الواقع تحمل هذه الطائرة على متنها 19 حاسوباً و80 معالجاً - صغرياً وكتبتم أنظمتها بست لغات برمجة مختلفة!

نتجت جميع الأخطاء الألفه الذكر عن أخطاء في البرمجة. في الحقيقة تعتبر البرمجيات - صنوعات بشرية ذات تعقيد عالٍ، فهي تقوم بوظائف كثيرة، ويطلب منها أن تحقق أهدافاً قد تكون في كثير من الأحيان متعارضة. كما أنها تتألف من مكونات عديدة معقدة في حد ذاتها. هذا عدا عن أن الممارسين البرمجية تتعرض أثناء مراحل التطوير لكثير من التغيير في المتطلبات بسبب تطور الاحتياجات وتطور السوق.

أما من حيث الكاف، فلو تأملت أسعار الأنظمة الحالية، لوجدت أن أسعار البرمجيات قد تجاوزت بكثير أسعار الأجهزة. ولو تابعت عملية التطوير البرمجي عن كثب، للاحظت أن الكاف المدفوعة في صيانة البرمجيات تفوق كفاف بنائها، حتى أن الدراسات قد بينت أن كفاف الصيانة يمكن أن تصل إلى عدة أضعاف كفاف البناء في الأنظمة التي تستمر في العمل (تعيش) لسنوات طويلة.

من الطبيعي إذاً أن نمعن النظر في العلم الذي يعنى ببناء هذه البرمجيات والذى يسمى هندسة البرمجيات، وما يحتويه من نظريات ومنهجيات وأدوات للمحترفين.

¹ مؤسسة حكومية تدعم الأفراد عند حدوث حوادث تتعلق بالأمن وتقدم لهم المعارف اللازمة المتعلقة بالأمن، وهو يتبع معهد هندسة البرمجيات.

2. أسس هندسة البرمجيات

تقوم هندسة البرمجيات على ممارسة عدد من الأنشطة.

1- وتعتبر النمجة modeling² أحد هذه الأنشطة الهامة. يلجأ مهندسو البرمجيات إلى النمجة ليتمكنوا من التعامل مع التعقيد الكبير للأنظمة.

2- كما تقوم هندسة البرمجيات على ممارسة نشاط آخر هو حل المسائل problem solving، إذ تستخدم النماذج التي توضع لإيجاد حل مقبول للمسألة المطروحة. يجري اختيار الحل من عدة خيارات بناء على خبرات المهندسين المكتسبة وتوجه الشروط المحددة للموارد (الوقت والموارد). هذا الاختيار.

3- إضافة إلى النشاطين السابقين، تعتمد هندسة البرمجيات على ممارسة نشاط آخر هو تحصيل المعرفة knowledge acquisition، فأنشاء عملية نمجة فضاء المسألة والحل يقوم المهندسون بجمع المعطيات، ثم يقومون بتجميعها وتنظيمها للحصول على المعلومات، ومن ثم تشكيلها ضمن معارف. ولا تتم عملية تحصيل المعرفة دفعة واحدة وعلى التتالي، فقد تؤدي معلومة لم تكن بالحسبان إلى هدم النموذج الموضوع كاملاً وإلى إعادة بنائه.

وأخيراً يمكننا أن نصف هندسة البرمجيات بأنها موجهة بالأسباب rationale driven. فأنشاء تحصيل المعرفة وأخذ القرارات المتعلقة بالنظام، يجب أن يحتفظ مهندس البرمجيات بالظروف والمبررات التي أدت به إلى هذه القرارات. إن التعبير عن هذه الأسباب التي دفعت إلى قرارات معينة ضمن نمذجة خاصة يسمح في حال طاب تعديلات على النظام ومراجعة القرارات السابقة - بمعرفة تأثير التعديلات المقترحة.

3. الأسئلة الأكثر انتشاراً في هندسة البرمجيات

مع انتشار البرمجيات وافتتاح اختصاصات جامعية في الكليات تعنى بتعليم العلوم التقنية الحديثة، كثرت الأسئلة والاستفسارات حول بعض المفاهيم والفروق الدقيقة فيما بينها، وإليك بعضاً من هذه الأسئلة:

1- ماذا نعني بالبرمجيات software؟

2- ماذا نعني بهندسة البرمجيات Software Engineering؟

3- ما الفرق بين هندسة البرمجيات Software Engineering وعلوم الحاسوب computer science؟

4- ما الفرق بين هندسة البرمجيات Software Engineering وهندسة النظام System Engineering؟

؟

² النمجة هي عملية تقتضي التركيز في مرحلة معينة على التفاصيل التي نهملها ونجاهل جميع التفاصيل الأخرى. وأثناء تطوير النظام البرمجي، يبني مهندسو البرمجيات عدداً كبيراً من النماذج models المختلفة للنظام ولمجال التطبيق.

- 5- ماذا نعني بالإجرائية البرمجية software process؟
- 6- ماذا نعني بنموذج الإجرائية البرمجية software process model؟
- 7- كيف تتوزع الكاف costs عند هندسة البرمجيات؟
- 8- ما هي طرائق هندسة البرمجيات software engineering methods؟
- 9- ما هي الأدوات الم-ساعدة في هندسة البرمجيات CASE (Computer-Aided Software Engineering)؟
- 10- ما هي خصائص البرمجيات الجيدة attributes of good software؟
- 11- ما هي أهم التحديات التي تواجه هندسة البرمجيات؟

سنرد في الفقرات التالية عن هذه الأسئلة لعل الصورة تصبح أوضح.

- 1- ماذا نعني بالبرمجيات software؟
- تستعمل كلمة برمجية أو برمجيات للدلالة على أحد المفاهيم التالية:
- 1- البرامج الحاسوبية والوثائق المرافقة لها مثل المتطلبات والنماذج التصميمية وأدلة المستخدم.
 - 2- المنتجات البرمجية التي قد تطوّر³ لزبون محدد أو للتوزيع في الأسواق. تلاحظ هنا أننا نميز بين نوعين من المنتجات:

- عمومية: بمعنى أنها قد طورت لتناسب احتياجات العديد من الزبائن المختلفين، ونضرب مثالا عنها البرمجيات المطورة للحواسيب الشخصية مثل Excel أو Word.
- مفصلة: بمعنى أنها قد طورت لتناسب احتياجات زبون وحيد بناء على المتطلبات التي يضعها.

ومن الجدير بالذكر هنا أنه يجري بناء البرمجيات الجيدة عبر بناء برامج جديدة أو تعديل شكل (Configure) نظم برمجية عمومية أو إعادة استخدام برمجيات موجودة.

- 2- ماذا نعني بهندسة البرمجيات Software Engineering؟
- هندسة البرمجيات هي الاختصاص الهندسي الذي يُعنى بجميع الجوانب المتعلقة بإنتاج البرمجيات. من الناحية التقنية يعرض هذا الاختصاص مجموعة التقنيات والمنهجيات والأدوات التي تساعد على بناء أنظمة برمجية عالية الجودة ضمن ميزانية معطاة وخلال فترة زمنية محددة، وذلك رغم التطورات الحاصلة باستمرار وأثناء التطوير.

- 3- ما الفرق بين هندسة البرمجيات Software Engineering وعلم الحاسوب computer science؟

³ إن كلمة تطوير هنا هي ترجمة للكلمة الإنكليزية development التي تعني إنشاء وبناء البرمجيات، وستستخدم في هذا المقرر بهذا المعنى وليس بمعنى إجراء تطوير على شيء موجود سابقا كما قد يخطر ببالك.

يهتم علم الحاسوب بدراسة النظريات والمنهجيات التي تشكل الأساس في بناء الأنظمة الحاسوبية والبرمجية. أما هندسة البرمجيات فتركز على الجوانب العملية المتعلقة ببناء البرمجيات المفيدة وتسليمها. إن الاطلاع على بعض المعارف المتعلقة بعلم الحاسوب مفيد لمهندس البرمجيات بنفسه المقياس الذي نجد فيه أن الاطلاع على بعض المعارف المتعلقة بالفيزياء مفيد لمهندس الإلكترونيات. ومع ذلك فإن نظريات علم الحاسوب لا تستطيع دوماً أن تعطي الحل لبعض المسائل المعقدة التي تحتاج إلى حل برمجي، وهنا يتم اللجوء إلى مقاربات "حسب الحالة" أثناء تطوير البرمجيات.

4- ما الفرق بين هندسة البرمجيات software engineering وهندسة النظم system engineering ؟

تهتم هندسة النظم بجميع الجوانب المتعلقة بتطوير النظم المعتمدة على الحاسوب computer-based systems بما فيها من تصميم وبناء للتجهيزات والبرمجيات ومن عناية بإجرائية التطوير. أما هندسة البرمجيات فهي ذلك الجزء من هذه الإجرائية الذي يهتم بتطوير البنية البرمجية الأساسية والتحكم والتطبيقات وبنى المعطيات التي تدخل في تكوين هذا النظام. وتعتبر هندسة النظم أقدم من هندسة البرمجيات فقد تمكن المهندسون وخلال الأعوام المائة الماضية من بناء وتجميع العديد من الأنظمة الضخمة كالمطارات والمعامل. ولكن مع تزايد نسبة البرمجيات في الأنظمة أصبح من الضروري الاهتمام بهذا الجزء بشكل خاص. وتقع مسؤولية تعريف المتطلبات وتصميم البنية العامة والتكامل ونشر كامل النظام بأجزائه البرمجية وبتجهيزاته على مهندس النظم.

5- ماذا نعني بالإجرائية البرمجية Software process ؟

الإجرائية البرمجية هي مجموعة الأنشطة التي تهدف إلى تطوير البرمجيات وتطويرها. ومهما اختلف المختصون على تفاصيل هذه الأنشطة، فإنهم متفقون على وجود الأنشطة العمومية الأربعة التالية:

1- التوصيف Specification: ويهدف إلى تحديد ما يجب على النظام فعله وشروط التطوير.

2- التطوير Development: ويهدف إلى إنتاج النظام البرمجي.

3- التحقق من الصلاحية Validation: وتهدف إلى التأكد من أن البرمجية الناتجة هي فعلاً الشيء الذي كان المستخدم ينشده.

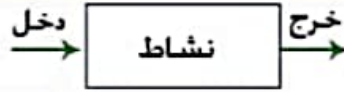
4- التطور Evolution: ويعنى بالتعديلات على البرمجية استجابة للتغيرات في المتطلبات.



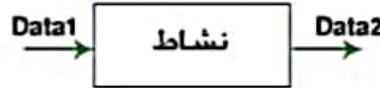
6- ماذا نعني بنموذج الإجرائية البرمجية Software process model ؟

نموذج الإجرائية البرمجية هو تمثيل مبسط للإجرائية البرمجية يجري عرضه من وجهة نظر معينة. ولهم المقصود من أن النموذج يعرض وجهة نظر معينة، نورد الأمثلة التالية:

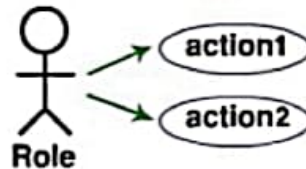
- 1 وجهة نظر تدفق العمل **Workflow perspective** (أو سير العمل): وفيها يمثل النموذج تتالي الأنشطة في الإجرائية مرفقة بالمدخلات والمخرجات التي ترافق هذه الأنشطة.



- 2 وجهة نظر تدفق المعطيات **Data-flow perspective**: وفيها يمثل النموذج تدفق المعلومات وما يطرأ عليها من تحولات بفعل الحواسيب أو البشر.



- 3 وجهة نظر الدور/الفعل **Role/action perspective**: وفيها يمثل النم- وذج دور الأش- خاص المعنيين بالإجرائية البرمجية والأنشطة التي تقع ضمن مسؤوليتهم.



وتوجد ثلاثة نماذج عمومية⁴ تعتمد عليها معظم نماذج الإجرائيات البرمجية، وهي:

- 1 المقاربة الشلالية **Waterfall approach**: تقوم بفصل الأنشطة آنفة الذكر بعضها عن بعض وترتيبها على مراحل متتالية، فتظهر لدينا المراحل التالية: توصيف المتطلبات **requirements** specification والتصميم **design** والتنفيذ **implementation** والاختبار **testing**.

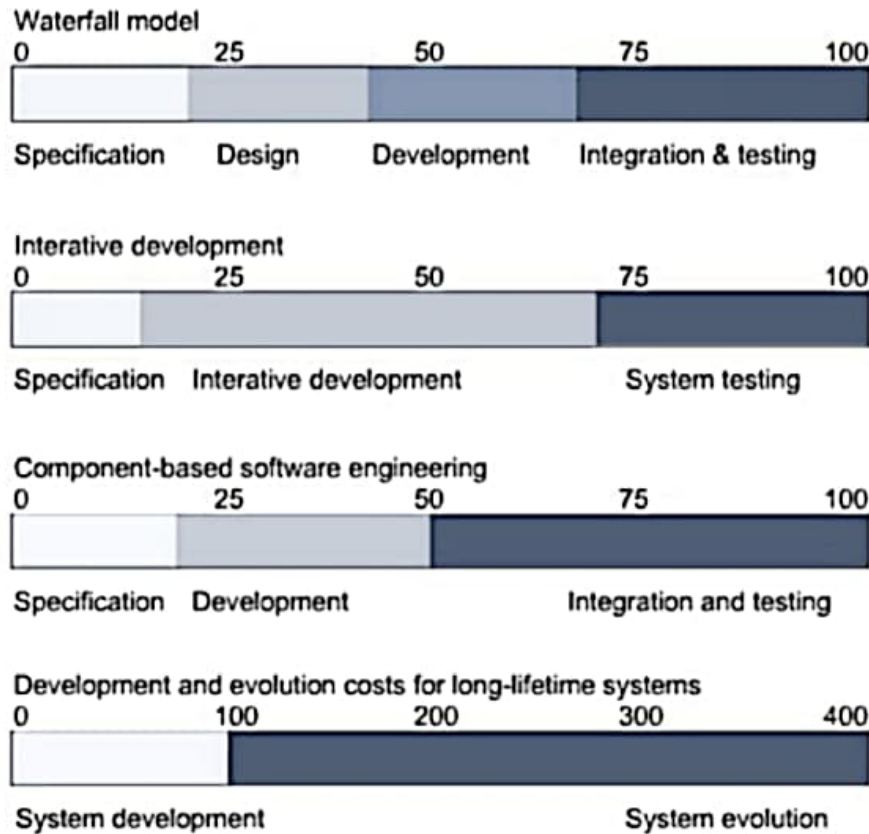
- 2 التطوير التكراري **Iterative development**: تعتمد هذه المقاربة على تدخل أنشطة التوصيف والتصميم والتنفيذ والاختبار. وهنا يمكن بناء نظام أول (ابتدائي) بسرعة باستخدام مواصفات موجزة للنظام المطلوب. يمكن العمل لاحقاً على تحسين النظام بالاستفادة من ملاحظات الزبون على هذا النظام الابتدائي للوصول به إلى الغاية المنشودة، أو يعاد بناء النظام من جديد باستخدام منهجيات أكثر احترافية للحصول على نظام صلب البنية وقابل للصيانة.

- 3 هندسة البرمجيات المعتمدة على المكونات **Component-based software engineering**: تقوم هذه المقاربة على فرضية أن بعض أجزاء النظام موجودة مسبقاً. وتركز إجرائية التطوير هنا على دمج هذه الأجزاء بدلاً من إعادة بنائها من الصفر.

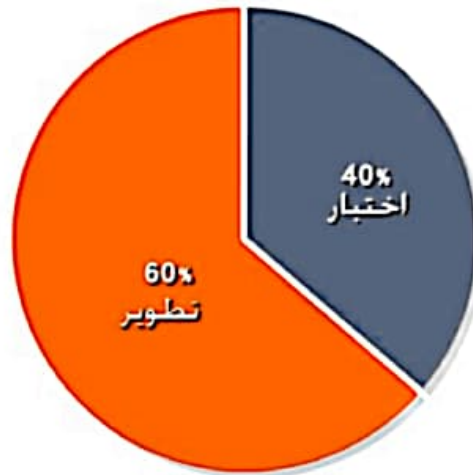
- 7 كيف تتوزع الكاف **costs** عند هندسة البرمجيات؟

⁴ لمزيد من التفاصيل يمكنك قراءة الفقرة 3.1. من الفصل الأول في مقرر تحليل وتصميم النظم وهي بعنوان التصميم البنيوي Structured Design.

نلاحظ على وجه الإجمال أن حوالي 60% من الكاف تصرف في مرحلة التطوير، وتصرف 40% من الكاف في مرحلة الاختبار. أما كيف تتوزع كلف التطوير على الأنشطة المختلفة، فيصعب إيجاد إجابة بسيطة على هذا السؤال نظراً لاختلاف الإجراءات المستخدمة وطبيعة البرمجيات قيد التطوير. فأنظمة الزمن الحقيقي تحتاج إلى إجراء عمليات تحقق من الصلاحية واختبار تفوق بكثير تلك المستخدمة في أنظمة الويب. كما أن الخصائص المطلوبة للنظام (كالأداء العالي والموثوقية) تؤثر كثيراً على هذا التوزيع. يمكننا في الشكل التالي ملاحظة اختلاف التوزيع باختلاف المقاربة والإجرائية المعتمدة.



كما نلاحظ أنه في حالة الأنظمة المفصلة لزبون محدد، تفوق الكاف اللازمة للتطوير تلك التي تصرف عند بناء (تطوير) البرمجية.



8- ما هي طرائق هندسة البرمجيات software engineering methods؟

في هندسة البرمجيات نطلق اسم طريقة software engineering method على مجموع نماذج النظ-ام وطرق التدوين والقواعد والنصائح التصميمية وتوجيهات الإجرائية التي تعتمد عليها مقارنة منهجية ما في تطوير البرمجيات. توجد العديد من الطرائق في هندسة البرمجيات ففي السبعينيات من القرن الماضي ظهرت الطرائق الموجهة بالوظيفة function oriented methods-ة طريقة-ة التحليل-ل البني-وي Structured Analysis التي وض-عها DeMarco وطريقة-ة JSD التي وض-عها Jackson، وفي الثمانينيات والتسعينيات من القرن نفسه ظهرت الطرائق الموجهة بالأغراض مد-ل طريقة-ة Booch و Rumbaugh. ولكل طريقة من هذه الطرق فلسفتها وتوصيفها ونماذجها المعتمدة، ومخططاتها البيانية وقواعدها وتوجيهاتها وترتيبها الخاص بها لأنشطة التطوير.

9- ما هي الأدوات المساعدة في هندسة البرمجيات CASE (Computer-Aided Software Engineering)؟

أنشئت الكلمة CASE من الأحرف الأولى لكلمات العبارة Computer-Aided Software Engineering ويندرج تحت هذه الكلمة عدد كبير من البرامج المستخدمة لدعم أنشطة إجرائية تط-وير البرمجيات. ومن المؤكد أنك استخدمت بعضاً منها فمحررات البرامج وبيئات التغليف debugging ه-ي ه-ن ه-ه هذه الأدوات. في الواقع يوجد لكل طريقة من طرائق هندسة البرمجيات أنظمة CASE تدعمها. وتنقسم هذه الأنظمة إلى قسمين:

- 1- أنظمة CASE التي تهتم بالمراحل العليا من التطوير (Upper-CASE): وهي الأدوات التي تدعم الأنشطة الأولى في إجرائية التطوير أي التوصيف والتصميم.
- 2- أنظمة CASE التي تهتم بالمراحل الدنيا من التطوير (Lower-CASE): وهي الأدوات التي تدعم الأنشطة الأخيرة في إجرائية التطوير أي البرمجة والتغليف والاختبار.

10- ما هي خصائص البرمجيات الجيدة attributes of good software؟

من المؤكد أن بناء البرمجيات عملية معقدة، تنتهي بتسليم الزبون البرمجية التي يتوقعها. ويفت-رض أن تؤدي البرمجية الوظائف التي طلبها الزبون. لكن يمكن لعدة مهندسين تطوير البرمجية المطلوبة مع وجود اختلاف كبير في الجودة فيما بين منتجاتهم. فما هي الخصائص التي تحدد جودة البرمجيات؟ يوجد العديد من الخصائص التي تسمح بتقييم جودة البرمجية وهي تسمى أي-ضاً بالمواص-فات غير الوظيفية؛ وهي خصائص هامة قد تؤثر في قبول البرمجية والتفاعل معها أو رفضها. تختل-ف ه-ه هذه الخصائص الحرجة من نظام لآخر ففي الأنظمة المصرفية يعتبر الأمان هو معيار الجودة، ف-ي ح-ين

تكون درجة الموثوقية في مقسم هاتفي هي المعيار الأهم، أما في الألعاب التفاعلية فإن سرعة استجابة النظام هي الأهم.

وقد جرى تعريف عدد من الخصائص تعتبر مميزات النظام البرمجي الجيد يمكن تعدادها فيما يلي:

1- **الصيانة Maintainability**: يجب أن تبنى البرمجية بحيث تكون قابلة للتطوير مع تغير

الاحتياجات وهو أمر يعد حتمياً في بيئة الأعمال اليوم.

2- **الاعتمادية Dependability**: تتضمن الاعتمادية عدداً من الخصائص كالموثوقية، والأمان

والأمان. وتكون البرمجية قابلة للاعتماد عليها إذا كان حدوث عطل فيها لا يتسبب بأذى اقتصادي أو فيزيائي.

3- **الفعالية Efficiency**: يجب أن تبنى البرمجية بحيث لا تفرط في موارد النظام كالمذاكرة أو

دورات المعالج بل تستخدمها بفعالية.

4- **مدى القبول Acceptability**: وهي تعكس مدى قبول المستخدم للبرمجية التي صممت له وهنا

يجب أن تكون البرمجية سهلة الاستخدام والفهم وقابلة للتكامل مع النظم الأخرى.

11- ما هي أهم التحديات التي تواجه هندسة البرمجيات؟

تتمثل أهم التحديات التي تواجه هندسة البرمجيات في القرن الواحد والعشرين في النقاط الثلاث

التالية: عدم التجانس والتسليم والثقة.

1- **تحدي عدم التجانس**: يزداد الطلب على البرمجيات التي تعمل في بيئة موزعة عبر الشبكات

التي تحتوي أنواعاً مختلفة من الحواسيب والأنظمة. يكمن هذا التحدي في ضرورة تطوير تقنيات تتيح بناء برمجيات تستطيع أن تتماشى مع بيئات التطوير والتنفيذ المختلفة.

2- **تحدي التسليم**: يستغرق تطبيق تقانات هندسة البرمجيات زمناً طويلاً في غالب الأحيان. ومع

تسارع إيقاع العمل والتغيرات السريعة المطلوبة، أصبح من الضروري تطوير تقنيات تتيح تسليم البرمجيات بزمن أقل دون التأثير على جودتها.

3- **تحدي الثقة**: مع دخول البرمجيات في جميع مناحي الحياة، أصبح من الضروري تطوير

تقنيات تثبت للمستخدم أنه يمكنه أن يثق بالبرمجيات.

4. المسؤولية المهنية والأخلاقية

تتطلب هندسة البرمجيات مسؤولية تتجاوز حدود تطبيق المهارات التقنية، إذ يجب أن يتصرف مهندس

البرمجيات بأمانة وخلق عال إذا كان يريد أن يكون محترماً مهنيًا. ونقصد بالسلوك الأخلاقي ما هو

أكثر من مجرد احترام القانون. فما هي أهم أصول المهنة؟

1- **السرية**.

يدخل المبرمج إلى شركة الزبون ويطلع على تفاصيل عمله وأسراره المهنية، لذا تقتضي أصول المهنة أن يحفظ هذه الأسرار.

2- الكفاءة.

تقتضي أصول المهنة أن يبذل مهندس البرمجيات قصارى جهده وأن يعمل وفق السوية المطلوبة. وهذا يعني كذلك أن لا يقدم على عمل يعرف مسبقاً أنه لا يملك المهارات الكافية له.

3- حقوق الملكية الفكرية.

تقتضي أصول المهنة أن يكون المهندس على اطلاع على القوانين المحلية التي تحكم استخدام الملكية الفكرية كبراءات الاختراع وحقوق النشر، وغيرها. ويجب أن يضمن المهندس حماية الملكية الفكرية لزيائنه.

4- عدم إساءة استخدام الحواسيب.

تقتضي أصول المهنة أن لا يستخدم مهندس البرمجيات مهاراته التقنية لإساءة استخدام حواسيب الآخرين. وقد تكون الإساءة بسيطة (كاللعب بالألعاب على حواسيب الزبـون) أو فاحشة (كنـشر الفيروسات).

وقد قامت مؤسستا ACM و IEEE بوضع القواعد الأخلاقية لممارسة المهنة. ويمكن الاطلاع عليها في الموقع www.acm.org/about/se-code. تتضمن هذه القواعد ثمانية بنـود تتعلق بـسلوك مهندس البرمجيات المحترف وبالقرارات التي عليه أن يأخذها. تتجه هذه البنـود لجميع المعنيين بهندسة البرمجيات من ممارسين ومعلمين ومدراء ومشرفين وصانعي سياسات ومتدربين وطلاب.

5. الإجرائية البرمجية

1- النماذج العمومية للإجرائية البرمجية

سبق أن ذكرنا في الفقرة 3.6. أنه يوجد ثلاثة نماذج للإجرائية البرمجية وهي:

النموذج الشلالي waterfall model

يتألف هذا النموذج من المراحل التالية:

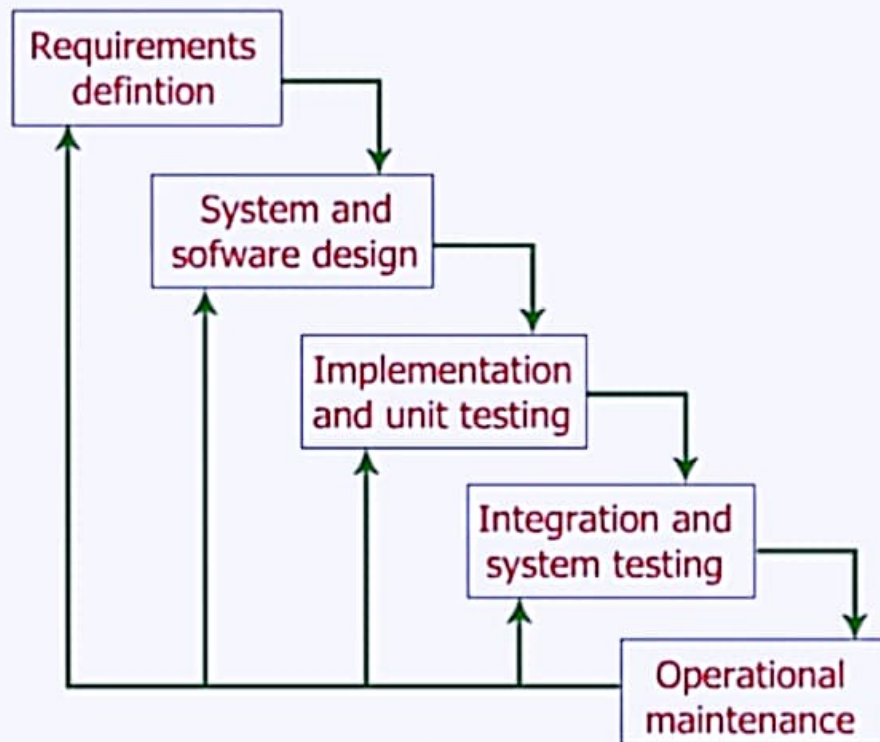
1- تحليل المتطلبات وتعريفها.

2- تصميم النظام والبرمجية.

3- التنفيذ والاختبارات الأحادية.

4- التكامل واختبار النظام.

5- التشغيل والصيانة.



إن الانتقاد الرئيسي الموجه لهذا النموذج هو صعوبة أخذ التغييرات المطلوبة بعين الاعتبار بعد أن تقلع الإجرائية، وذلك لأننا يجب أن ننتهي تماماً من مرحلة ما قبل الانتقال إلى المرحلة التالية. لهذا السبب، يناسب هذا النموذج الأنظمة التي تكون متطلباتها واضحة جداً والتغييرات المتوقعة على هذه المتطلبات محدودة أثناء مرحلة التصميم.

يستخدم هذا النموذج في المشاريع الهندسية المتعلقة بالأنظمة الكبيرة حيث يجب بناء النظام في مواعيد متعددة.

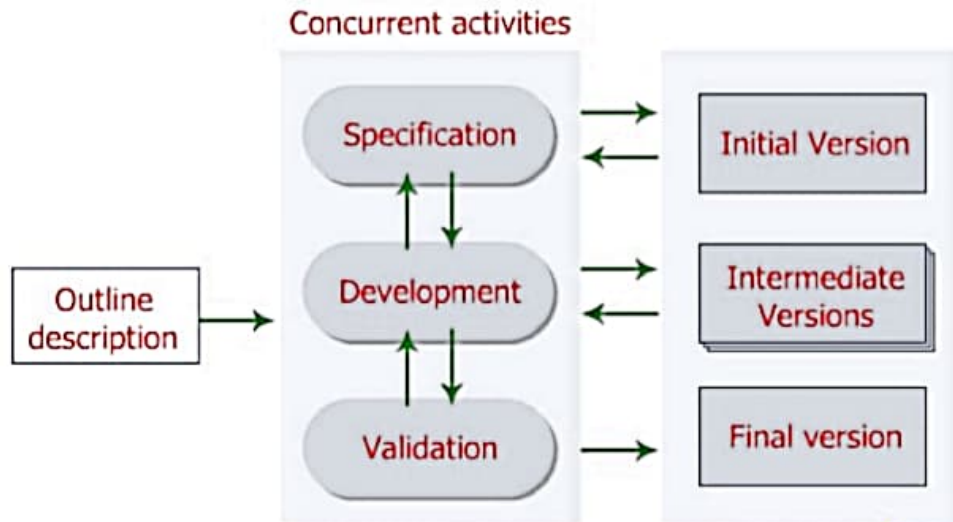
يمكن أن يستخدم هذا النموذج في حالتين:

1- حالة التطوير الاستكشافي Exploratory development.

في هذه الحالة يبدأ العمل مع الزبون انطلاقاً من توصيف للخطوط العريضة للنظام -ام به- دف التطور تدريجياً للوصول إلى النظام النهائي. وتكون البداية عادة مع متطلبات واضحة ومفهومة، ثم يضاف إلى النظام الخصائص الجديدة عندما يقترحها الزبون.

2- حالة النمجة الأولية throw-away prototyping.

تحدث هذه الحالة عندما يكون الزبون غير قادر على توصيف النظام، فيجري بناء نموذج أولي يتفق من خلاله على المواصفات المطلوبة للنظام. وبعد فهم المواصفات يُرمى النموذج الأولي ثم يبنى النظام.



النموذج التطوري evolutionary model

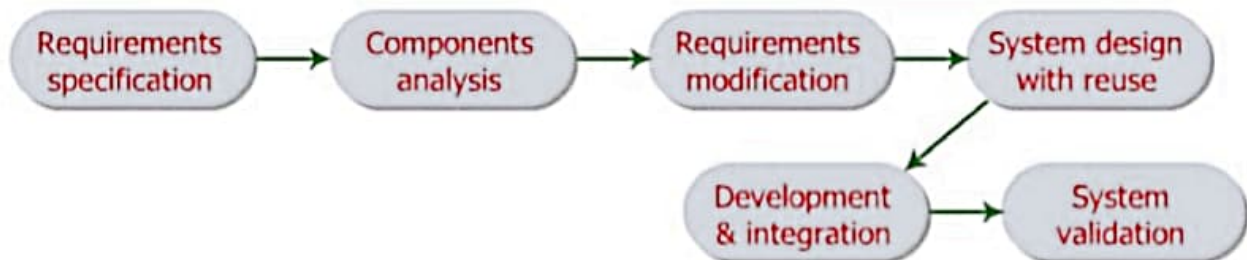
إن الانتقادات الرئيسية الموجهة لهذا النموذج تكمن في عدم وضوح الإجراءات ومراحلها، وفي الحصول على نظام ذي بنية مخلقة، إضافة إلى أن بناء النموذج الأولي يتطلب مهارات وإتقان لغات خاصة (مثل Visual Basic).

يناسب هذا النموذج الأنظمة التفاعلية الصغيرة أو متوسطة الحجم، كما يمكن أن يستخدم في بعض أجزاء الأنظمة الكبيرة كالجزء المتعلق ببناء واجهات المستخدم، أو في النظم التي تكون مدة حياتها قصيرة.

هندسة البرمجيات المعتمدة على المكونات components based software Engineering

تعمل هذه الإجرائية وفق المراحل التالية:

- 1- توصيف المتطلبات اللازمة للنظام.
- 2- تحليل المكونات الموجودة في السوق ومدى تحقيقها للمتطلبات الخاصة بالنظام.
- 3- تعديل المتطلبات بما يتوافق مع المكونات الموجودة⁵.
- 4- تصميم النظام مع إعادة استخدام المكونات الموجودة.
- 5- التطوير والتكامل بين المكونات.
- 6- التحقق من صلاحية النظام.



⁵ ينسق المهندس مع الزبون لضمان موافقته على التعديلات.

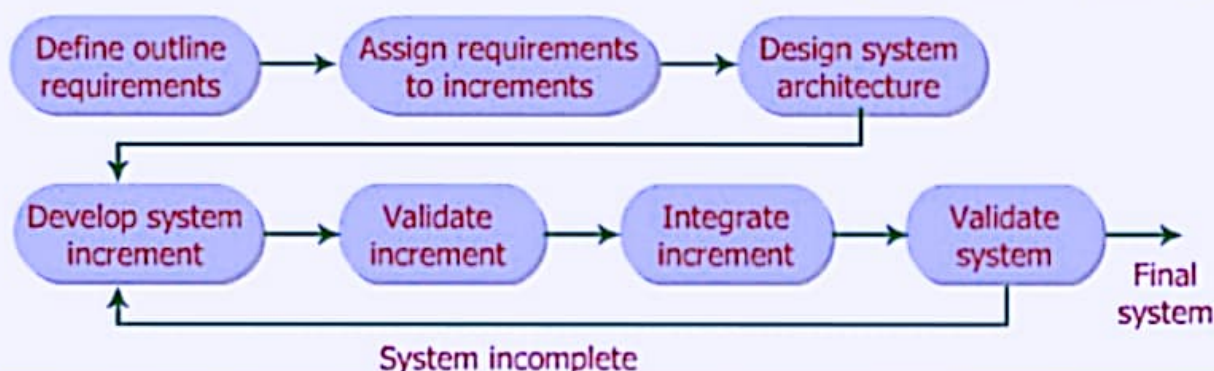
سبق وأشرنا إلى أننا في هذا النموذج من الإجرائية البرمجية نقوم بعملية "إعادة استخدام" reuse لمكونات جاهزة (COTS (Commercial-off-the-shelf يجري إجراء التكامل في بينها لبناء النظام.

2- التكرار في الإجرائية

تتطور متطلبات الأنظمة تطوراً مستمراً أثناء سير العمل في المشروع، وعليه فإن التكرار يعتبر جزءاً لا يتجزأ من الإجرائية، حيث علينا تكرار العمل على المراحل السابقة دوماً في الأنظمة الكبيرة. ويطبق هذا التكرار على أي نموذج من النماذج العمومية للإجرائية. يمكن إجراء هذا التكرار باستخدام إحدى المقاربتين التاليتين:

التسليم التزايدى incremental delivery

بدلاً من تسليم النظام دفعة واحدة، يُجزأ التطوير والتسليم إلى تزايدات increments حيث يتم ذلك من خلال تزايد جزءاً من الوظائف المطلوبة. لتحقيق هذا الأمر في الإجرائية، تُرتب متطلبات المستخدم حسب الأولويات التي يحددها المستخدم، ثم يبدأ العمل على المتطلبات ذات الأولويات الأعلى. وبمجرد أن يبدأ تطوير التزايد الذي يحقق هذه المتطلبات "تجمد" المتطلبات المتعلقة به في حين يبقى من الممكن تغيير المتطلبات المتعلقة بالتزايدات التالية.

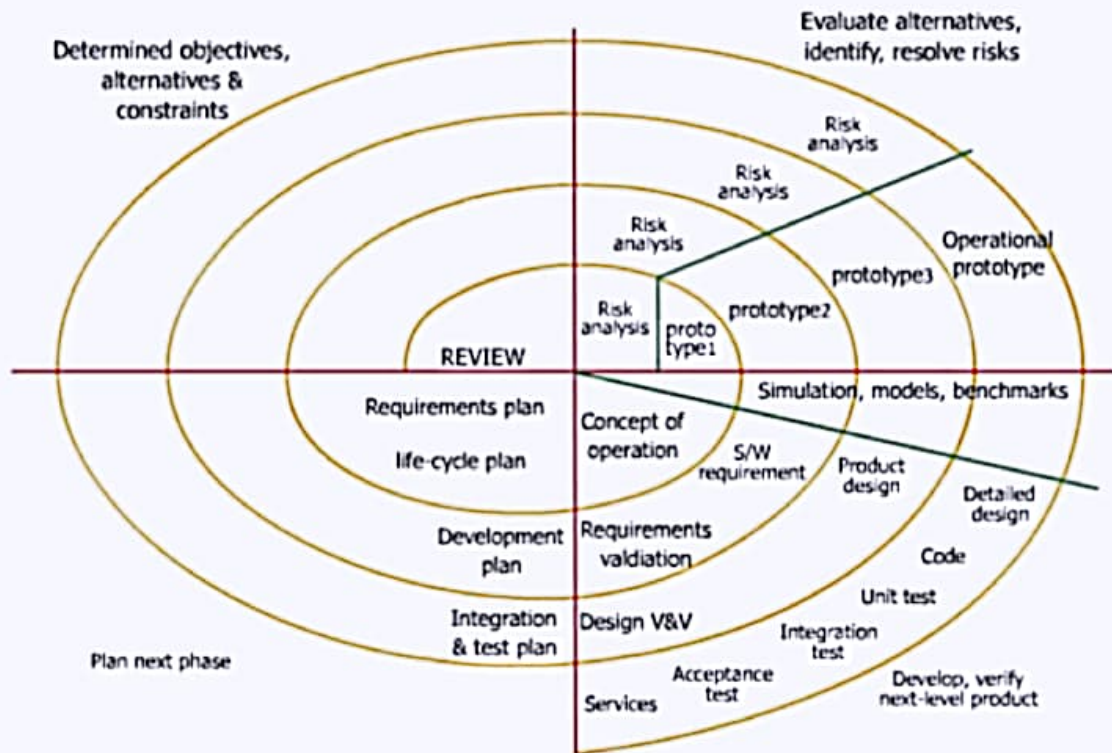


يتمتع هذا التطوير التزايدى بميزات عدة، إذ يبدأ العمل على أكثر ما يهم المستخدم وهكذا يحصل في وقت مبكر على إصدار من النظام يلبي حاجاته الملحة. كما أن كل تزايد يعتبر بمثابة "نموذج أولي" prototype يسمح بإيضاح المطلوب للتزايدات اللاحقة.

إن العمل وفق هذه الآلية يحمي النظام من الفشل، لاسيما وأن الخدمات ذات الأولوية العالية تحظى بالنصيب الأكبر من الاختبار.

التطوير الحلزوني Spiral development

في هذه المقاربة يجري تمثيل الإجرائية بشكل حلزوني بدلاً من النظر إليها ككتال للأنشطة مع إمكانية العودة إلى الوراء. تمثل كل دورة في الحلزون مرحلة من مراحل الإجرائية. لا توجد في هذه المقاربة مراحل مثبتة (كالتحليل أو التصميم)، وإنما يجري اختيار الدورات في الحلزون حسب الحاجة. وهذا يجري تقدير المخاطرة وكيفية التعامل معها ضمن الإجرائية.



قطاعات النموذج الحازوني:

- 1- تحديد الأهداف: يجري تحديد أهداف واضحة ومعينة للمرحلة.
- 2- تقدير المخاطرة وإضعافها: يجري تقدير المخاطر وتحدد الأنشطة التي ستخفض من أهـم هذه المخاطر.
- 3- التطوير والتحقق من الصلاحية: يتم اختيار نموذج التطوير من بين النماذج العمومية الـ ثلاث التي رأيناها آنفاً
- 4- التخطيط: تتم مراجعة المشروع والتخطيط للمرحلة التالية من الحازون.

3- أنشطة الإجرائية

سبق أن ذكرنا في الفقرة 3.5. أن الأنشطة التي تتألف منها الإجرائية هي:

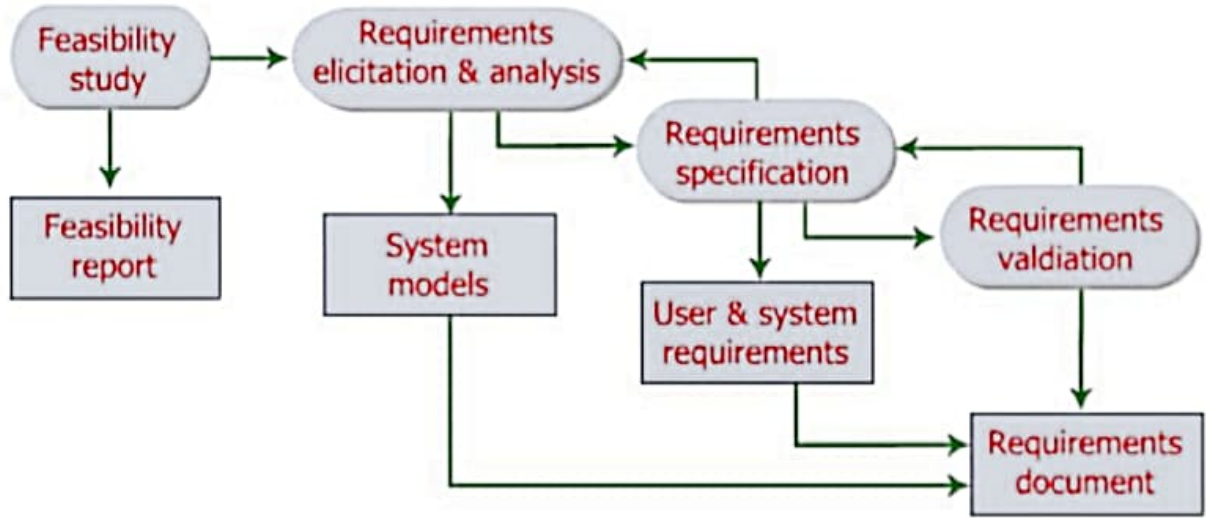
توصيف البرمجيات

هي الإجرائية التي تسمح بتحديد الخدمات المطلوبة من النظام، والقيود التي تقيد تطويره و- شغيله. إن هذا النشاط يشكل ما يسمى "هندسة المتطلبات" **requirement engineering**. تتألف إجرائية هندسة المتطلبات من الخطوات التالية:

- 1- دراسة الجدوى.
- 2- تجميع المتطلبات وتحليلها.
- 3- توصيف المتطلبات.

4- التحقق من صلاحية المتطلبات.

يبين الشكل هذه الإجرائية، كما تظهر الوثائق الناتجة عن هذه الخطوات على الشكل (ضمن مستطيل).



تصميم البرمجيات وتنجزها

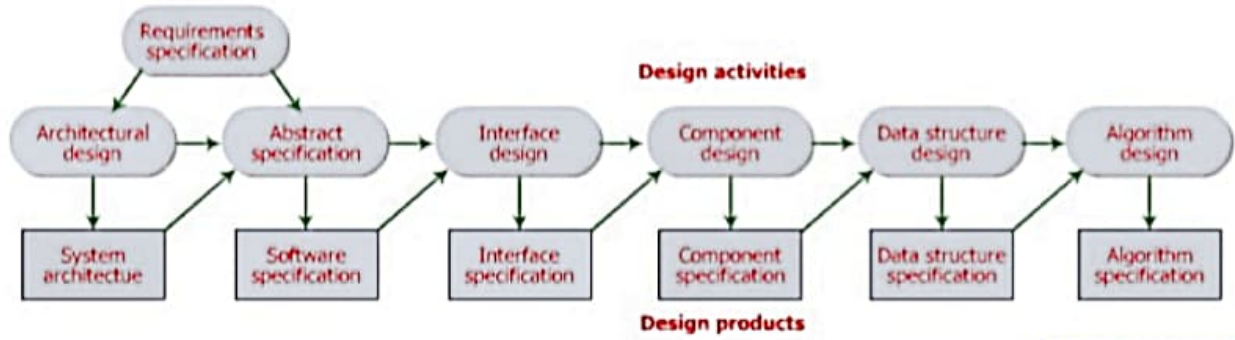
في هذا النشاط نقوم بإجرائية تهدف إلى تحويل مواصفات النظام التي حددناها في المرحلة السابقة إلى نظام قابل للتشغيل. ونلاحظ هنا أننا نتحدث عن نشاطين هما التصميم والتنجز. يهدف التصميم إلى تصميم بنية البرمجية التي تحقق المواصفات، في حين يهدف التنجز إلى ترجمة هذه البنية إلى برنامج قابل للتشغيل وكثيراً ما تتداخل الأنشطة المتعلقة بالتصميم والتنجز فيما بينها.

أنشطة إجرائية التصميم

تتألف إجرائية التصميم من الأنشطة التالية:

- 1- تصميم البنيان.
- 2- التوصيف الموجز.
- 3- تصميم الواجهات.
- 4- تصميم المكونات.
- 5- تصميم بنى المعطيات.
- 6- تصميم الخوارزميات.

يبين الشكل هذه الإجرائية، كما تظهر الوثائق الناتجة عن هذه الخطوات على الشكل (ضمن مستطيل).



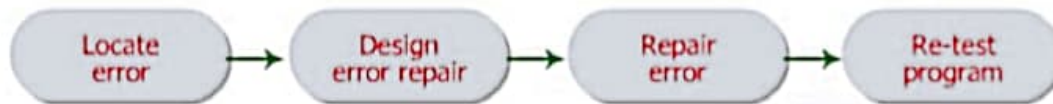
الطرائق البنيوية

هي مقاربات منهجية لتطوير وبناء تصميم للبرمجية. ويعبر عن التصميم عادة بمجموعة من النمذجات البنيوية. تتنوع النماذج بتنوع وجهة النظر والفلسفة المعتمدة في بنائها وفيما يلي بعض أنواع النمذجات المستخدمة:

- 1- نموذج الأغراض.
- 2- نموذج التالي.
- 3- نموذج الانتقال بين الحالات.
- 4- النموذج البنيوي.
- 5- نموذج تدفق المعطيات.

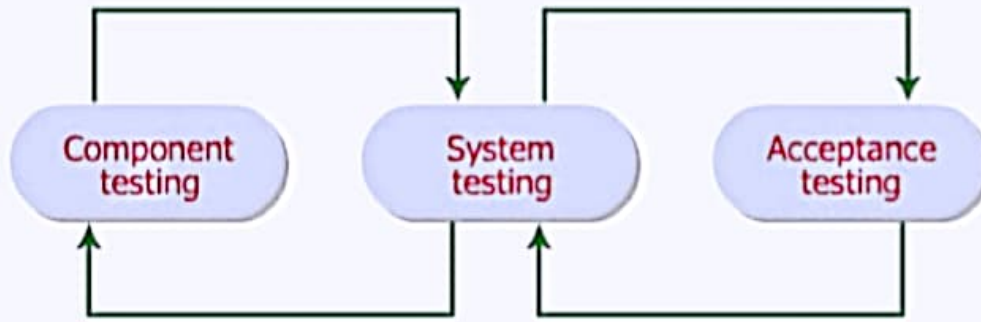
البرمجة والتفلية

نكرنا في بداية هذه الفقرة أن التنجيز يهدف إلى ترجمة التصميم إلى برنامج، وهنا علينا القيام بالبرمجة وبفحص البرنامج لإزالة جميع الأخطاء البرمجية التي قد يقع فيها المبرمج. إن عملية البرمجة نشاط شخصي إذ لا يوجد إجرائية عمومية للبرمجة. ويجري المبرمج عادة عدداً من الاختبارات للكشف عن الأخطاء التي يمكن أن يكون قد ارتكبها وإصلاحها وهذا ما يسمى إجرائية التفلية debugging. يبين الشكل إجرائية التفلية.



التحقق من صلاحية البرمجيات

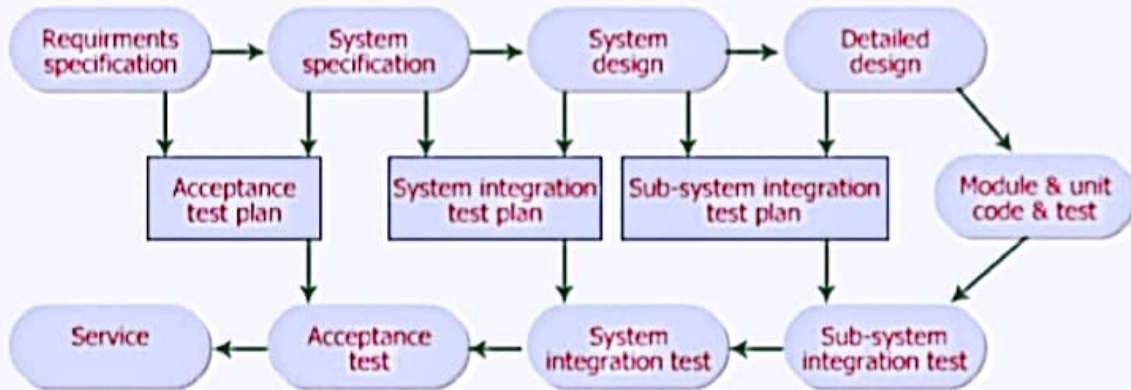
تهدف عملية التحقق من الصلاحية validation والصحة verification (وهو ما يرمز له بـ V&V) إلى إثبات أن النظام يحقق المواصفات التي بني انطلاقاً منها وأنه يحقق متطلبات الزبون. وتتضمن هذه العملية إجرائيات تدقيق ومراجعة واختبارات للنظام. يبين الشكل إجرائية الاختبار.



لاختبار النظام يجري إعداد عدد من حالات الاختبار المستنتجة من مواصفات المعطيات الحقيقية التي سيعالجها النظام لاحقاً، ثم يتم تنفيذ البرنامج باستخدام هذه الحالات.

تتألف إجرائية الاختبار من عدة خطوات:

- 1- اختبار المكونات (أو ما يسمى الاختبارات الأحادية): حيث تختبر المكونات كل على حدة. ونقصد بالمكونات هنا التوابع أو الأغراض أو أية تجمع ممتاسك من هذه الكيانات.
- 2- اختبار النظام: حيث يختبر النظام ككل لاسيما خصائص الجودة كالأداء والجودة والأمن.
- 3- اختبارات القبول: حيث يختبر النظام باستخدام معطيات الزبون للتحقق من أن النظام يحمي - كما يحق - احتياجات هذا الزبون. يبين الشكل مراحل الاختبار.



تطور البرمجيات

إن الطبيعة المتغيرة للبرمجيات تجعل منها منتجات قابلة للتغيير. ونظراً للتغيرات المستمرة في عالم الأعمال فإن متطلبات الزبون تكون هي كذلك في تغير مستمر وبالتالي يجب أن ينتقل هذا التغيير إلى البرمجية التي تدعم الأعمال. وقد حاول العاملون في هذا المجال وضع حد فاصل بين التطوير (البناء) والتطور (الصيانة)، إلا أن هذا الأمر لم يحقق نجاحاً نظراً لأن معظم البرمجيات التي يجري العمل عليها هي بحد ذاتها برمجيات متطورة لأخرى سبقتها، لأن عدد البرمجيات الجديدة أخذ في التزايد. يبين الشكل كيف يجري التطور على النظام البرمجية.

